CMPS 3240 Comp Arch II: Organization
Homework 8

Noah Gallego

May 9, 2026

4.11

Consider the following loop.

```
loop:lw  r1,0(r1)
     and r1,r1,r2
     lw  r1,0(r1)
     lw  r1,0(r1)
     beq r1,r0,loop
```

Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, and that the pipeline has full forwarding support. Also assume that many iterations of this loop are executed before the loop exits.

4.11.1

Show a pipeline execution diagram for the third iteration of this loop, from the cycle in which we fetch the first instruction of that iteration up to (but not including) the cycle in which we can fetch the first instruction of the next iteration. Show all instructions that are in the pipeline during these cycles (not just those from the third iteration).

I3′, I4′, I5′ are from iteration 2; I1–I5 are from iteration 3. `--` = bubble; repeated entry = stall.

Stage	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
IF	I1	I1	I2	I3	I3	I4	I5	I5
ID	I5′	I5′	I1	I2	I2	I3	I4	I4
EX	I4′	–	I5′	I1	–	I2	I3	–
MEM	–	I4′	–	I5′	I1	–	I2	I3
WB	I3′	–	I4′	–	I5′	I1	–	I2

4.12

This exercise is intended to help you understand the cost/complexity/performance trade-offs of forwarding in a pipelined processor. Problems in this exercise refer to pipelined datapaths from

Figure 4.45. These problems assume that, of all the instructions executed in a processor, the following fraction of these instructions have a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the instruction that consumes the result (1st instruction that follows the one that produces the result, 2nd instruction that follows, or both). We assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle, so "EX to 3rd" and "MEM to 3rd" dependences are not counted because they cannot result in data hazards. Also, assume that the CPI of the processor is 1 if there are no data hazards.

EX to 1st Only	**MEM to 1st Only**	**EX to 2nd Only**	**MEM to 2nd Only**	**EX to 1st and MEM to 2nd**	**Other RAW Dependences**
5%	20%	5%	10%	10%	10%

Assume the following latencies for individual pipeline stages. For the EX stage, latencies are given separately for a processor without forwarding and for a processor with different kinds of forwarding.

IF	**ID**	**EX (no FW)**	**EX (full FW)**	**EX (FW from EX/MEM only)**	**EX (FW from MEM/WB only)**	**MEM**	**WB**
150 ps	100 ps	120 ps	150 ps	140 ps	130 ps	120 ps	100 ps

4.12.1

If we use no forwarding, what fraction of cycles are we stalling due to data hazards?

Stalls/inst $= 0.05(2) + 0.20(2) + 0.05(1) + 0.10(1) + 0.10(2) = 0.85$.

$$\frac{0.85}{1.85} \approx \mathbf{45.9}\%$$

4.12.2

If we use full forwarding (forward all results that can be forwarded), what fraction of cycles are we staling due to data hazards?

Only MEM-to-1st (load–use) stalls remain. Stalls/inst $= 0.20$.

$$\frac{0.20}{1.20} \approx \mathbf{16.67}\%$$

4.12.3

Let us assume that we cannot afford to have three-input Muxes that are needed for full forwarding. We have to decide if it is better to forward only from the EX/MEM pipeline register (next-cycle forwarding) or only from the MEM/WB pipeline register (two-cycle forwarding). Which of the two options results in fewer data stall cycles?

EX/MEM only: $0.05(0) + 0.20(2) + 0.05(1) + 0.10(1) + 0.10(1) = 0.65$ stalls/inst.
MEM/WB only: $0.05(1) + 0.20(1) + 0.05(0) + 0.10(0) + 0.10(1) = 0.35$ stalls/inst.

MEM/WB only (two-cycle forwarding) gives fewer stalls.

4.12.4

For the given hazard probabilities and pipeline stage latencies, what is the speedup achieved by adding full forwarding to a pipeline that had no forwarding?

Clock = 150 ps in both cases.

$$\text{Speedup} = \frac{1.85 \times 150}{1.20 \times 150} = \frac{1.85}{1.20} \approx \mathbf{1.54}\times$$

4.12.5

What would be the additional speedup (relative to a processor with forwarding) if we added time-travel forwarding that eliminates all data hazards? Assume that the yet-to-be-invented time-travel circuitry adds 100 ps to the latency of the full-forwarding EX stage.

CPI = 1, clock = 250 ps.

$$\text{Speedup} = \frac{1.20 \times 150}{1.0 \times 250} = \frac{180}{250} = \mathbf{0.72}\times \text{ (slower)}$$

4.12.6

Repeat 4.12.3 but this time determine which of the two options results in shorter time per instruction.

Clock = 150 ps for both.

$$T_{\text{EX/MEM}} = 1.65 \times 150 = 247.5 \text{ ps}, \qquad T_{\text{MEM/WB}} = 1.35 \times 150 = 202.5 \text{ ps}$$

MEM/WB only gives shorter time per instruction.

4.15

The importance of having a good branch predictor depends on how often conditional branches are executed. Together with branch predictor accuracy, this will determine how much time is spent stalling due to mispredicted branches. In this exercise, assume that the breakdown of dynamic instructions into various instruction categories is as follows:

R-Type	**BEQ**	**JMP**	**LW**	**SW**
40%	25%	5%	25%	5%

Also, assume the following branch predictor accuracies:

Always-Taken	**Always-Not-Taken**	**2-Bit**
45%	55%	85%

4.15.1

Stall cycles due to mispredicted branches increase the CPI. What is the extra CPI due to mispredicted branches with the always-taken predictor? Assume that branch outcomes are determined in the EX stage, that there are no data hazards, and that no delay slots are used.

Mispredict penalty = 2 cycles.

$$\Delta\text{CPI} = 0.25 \times 0.55 \times 2 = \mathbf{0.275}$$

4.15.2

Repeat 4.15.1 for the "always-not-taken" predictor.

$$\Delta\text{CPI} = 0.25 \times 0.45 \times 2 = \mathbf{0.225}$$

4.15.3

Repeat 4.15.1 for for the 2-bit predictor.

$$\Delta\text{CPI} = 0.25 \times 0.15 \times 2 = \mathbf{0.075}$$

4.15.4

With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaces a branch instruction with an ALU instruction? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.

Branches halve to 12.5%; instruction count unchanged.

$$\text{Speedup} = \frac{1.075}{1 + 0.125 \times 0.15 \times 2} = \frac{1.075}{1.0375} \approx \mathbf{1.036}\times$$

4.15.5

With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaced each branch instruction with two ALU instructions? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.

Instruction count $\to 1.125N$; branches $= 1/9$.

$$\text{Speedup} = \frac{1.075\,N}{1.125N \times (1 + \frac{1}{9} \times 0.15 \times 2)} = \frac{1.075}{1.1625} \approx \mathbf{0.925}\times \text{ (slower)}$$

4.15.6

Some branch instructions are much more predictable than others. If we know that 80% of all executed branch instructions are easy-to-predict loop-back branches that are always predicted correctly, what is the accuracy of the 2-bit predictor on the remaining 20% of the branch instructions?

$$0.80(1) + 0.20\,x = 0.85 \implies x = \mathbf{25}\%$$